Architecture and User Feedback

Production architecture, guardrails, routing, caches, observability, orchestration, and feedback design

Build from a simple evaluated workflow, add components only for demonstrated needs, and make failures observable. Use user feedback carefully: it is valuable data, but it is biased, contextual, and privacy-sensitive.

Source: Chip Huyen, AI Engineering: Building Applications with Foundation Models, first edition (2025). Source: book pp. 449–494 / PDF p. 473. These are original study notes, not verbatim chapter reproductions. Tool names, prices, model comparisons, and research findings reflect the book's publication period unless stated otherwise.

01 AI Engineering Architecture

Source: book pp. 449–465 / PDF p. 473

Step 1: Enhance context

Begin with user request → model → answer, then add retrieval/tools when missing information limits usefulness. Sources can include documents, tables, images, user-authorized history, and external services. Context construction plays a role analogous to feature engineering.

Read-only actions, relevance, freshness, permissions, and source provenance belong in this design. More context is not always better; evaluate what the model can use and how added retrieval affects latency.

Step 2: Put in guardrails

Input guardrails detect unauthorized sensitive data, out-of-scope requests, and adversarial prompts. Apply privacy checks to the final context after retrieval as well as the raw user message. Local tokenization/redaction with a protected reverse map can support permitted masked-data workflows, but is not a universal privacy guarantee.

Output guardrails detect malformed, unsupported, unsafe, or inappropriate answers and define what happens: repair, bounded retry, fallback, abstention, or human handoff. Measure false refusals and unsafe misses. Guardrails add cost and may conflict with immediate streaming; buffering or incremental checks need explicit product decisions. Consequential actions need trusted authorization, not just output scoring.

Step 3: Model router and gateway

A router chooses the right destination: smaller specialist, larger model, tool path, fixed answer, or human. Intent/scope classification can happen before retrieval to avoid unnecessary work. Context length, capability, policy, budget, and current availability constrain decisions.

A gateway standardizes access to internal and external models, centralizes authentication/authorization, model catalogs, usage limits, cost accounting, fallbacks, and logging. Keep provider quirks visible enough to debug. Router and gateway responsibilities overlap in some platforms but are conceptually distinct: choose the path versus manage access and execution.

Step 4: Reduce latency with caches

Exact caching reuses results for equivalent keys; semantic caching retrieves answers to meaningfully similar queries. Cache embeddings, retrieval results, or final responses where appropriate. KV/prefix caches at the inference layer are different from these system-level caches.

Semantic similarity can hide differences in identity, time, permissions, or constraints, so a high vector score alone is insufficient. Include relevant context and versions in keys, isolate tenants/users, set TTLs, and invalidate stale content. LRU, LFU, and FIFO govern eviction. Evaluate actual hit rate, answer quality, memory, and lookup overhead before adding a semantic cache.

Step 5: Add agent patterns

Feed observations or unsatisfactory intermediate answers back into retrieval/planning when the task needs multiple steps. Add write tools only with validated arguments, least privilege, approvals, and recoverable execution.

Set step, token, retry, deadline, and spend limits. Each extra model, router, tool, or loop creates new failure modes and observability needs. A deterministic workflow is often preferable when the sequence is already known; dynamic agents should justify their added complexity.

---

02 Monitoring and Observability

Source: book pp. 465-472 / PDF p. 489

Detection, diagnosis, and operational metrics

Monitoring tracks defined signals; observability instruments the system so unexpected failures can be investigated. Track failures, attacks, drift, and opportunities for quality/cost improvement from the start.

MTTD: average time to detect an incident. **MTTR:** time to respond/resolve under the team's definition. **Change failure rate:** fraction of deployments causing failures that need remediation, not fraction of failed requests.

Connect offline criteria with live signals: grounding, formatting, refusals, safety, retrieval quality, completion, latency tails, tokens, cost, rate limits, and business outcomes.

Metrics, logs, traces, and drift

Metrics detect trends and spikes; logs record events; traces link one request across routing, retrieval, generation, judging, and tools. Record model/endpoint/version, prompt/template, sampling, cache decisions, source identifiers, tool calls/results, latency, and errors so regressions are explainable.

Logging must respect redaction, access, retention, and data-minimization requirements; "log everything" is not permission to store secrets. Protect any raw payloads and use approved sampling when exhaustive capture is costly.

Detect prompt/template changes, provider/model drift, user behavior shifts, source-data changes, and tool-version changes. Feed new failures back into evaluation and track deploy versions to distinguish causes.

---

03 AI Pipeline Orchestration

Source: book pp. 472–474 / PDF p. 496

Components, chaining, and framework selection

Orchestration defines components and their control flow: query processing, retrieval, prompt assembly, generation, checks, and handoff. It may support parallel branches, conditions, loops, persistence, and recovery.

Evaluate integration/extensibility, support for genuinely complex pipelines, usability, scalability, and performance. A framework can accelerate development while hiding prompts and state. Start with simple explicit functions when adequate; adopt an orchestrator when the operational benefits outweigh abstraction and dependency costs.

---

04 User Feedback and Conversational Signals

Source: book pp. 474–480 / PDF p. 498

Explicit and implicit feedback

Explicit signals include ratings, comments, and pairwise choices. Implicit signals include edits, acceptance, retries, early exits, conversation behavior, and downstream use. Feedback can inform evaluation, product decisions, personalization, and approved training data.

Users must know how their data is used. Consent, ownership, retention, access, and deletion apply to feedback just as to other user data. Do not equate feedback collection with permission to train automatically.

Extracting natural-language feedback

Corrections such as "I meant..." reveal misunderstandings. Rephrasing, requests to verify sources, abandonment, and negative sentiment may reveal uncertainty or friction. Direct edits can provide strong preference demonstrations but still require correctness checks.

Model refusals and the user's response to them need interpretation: some refusals are required and appropriate. A request to "check again" does not establish that the answer was wrong. Track context and reasons rather than treating isolated phrases as infallible labels.

Other conversational feedback

Conversation length, topic diversity, continued use, saving/sharing/deleting outputs, and renaming conversations can offer signals. Meaning is product-dependent: long companionship sessions may be desirable, while long support sessions may mean unresolved problems.

Integrated tools can observe edits and task use more directly than standalone chat products. Acceptance may reflect convenience rather than correctness. Distinguish satisfaction with format, usefulness, factuality, and outcome whenever the interaction allows it.

---

05 Feedback Design

Source: book pp. 480–490 / PDF p. 504

When to collect feedback

Keep a nonintrusive route to report problems throughout the user journey. Initial preferences help cold start; targeted feedback is valuable after errors, at uncertainty points, and after meaningful successes.

Offer focused edits rather than forcing full regeneration when a small correction suffices. Pairwise drafts can clarify style preferences, but should not ask users to resolve technical correctness they cannot judge. Ask only when the information gained justifies interrupting the workflow.

How to collect it

Use clear, low-effort options tied to genuine actions: accept, edit, reject, regenerate, choose a candidate, or report a specific problem. Include "neither," "unsure," or "I don't know" where appropriate. Test icon direction, label wording, scales, accessibility, and whether users interpret them correctly.

Decide whether feedback is private or visible to others. Public signals change behavior and can discourage candor; private signals have their own limitations. Balance opportunities for positive and negative feedback, and avoid rewarding only easily captured signals.

---

06 Feedback Limitations

Source: book pp. 490–494 / PDF p. 514

Bias, randomness, and degenerate loops

Users can click randomly, prefer the first or most recent option, interpret scales differently, or favor pleasing wording over truth. Frequent users and readily measurable actions can be overrepresented. Human feedback is valuable but not automatically a correct target.

Exposure creates a loop: popular outputs are shown more, gain more feedback, and become still more dominant. The product's focus and user population may narrow together. Preference optimization can encourage sycophancy and suppress diversity.

Audit subgroup and tail performance, preserve exploration, calibrate against correctness/expert evidence, and separate personalization from global behavior changes. Evaluate whether the feedback-driven update actually benefits users.

Revision Check

1. Where do privacy checks belong after context augmentation?
2. What distinguishes a router from a gateway?
3. How would cache keys prevent cross-user leakage?
4. Which trace fields explain a wrong final answer?
5. When does a longer conversation mean the system is getting worse?

A dependable AI product combines evaluated components, trusted action boundaries, useful observability, and a governed feedback loop. Add complexity deliberately and keep the user's real outcome in view.

Book-based notes, separate from the earlier transcript-based notes. Coverage is indexed by section in coverage.md. Source links open the supplied book at the relevant PDF page. Detailed examples, citations, and complete experimental evidence remain in the book.